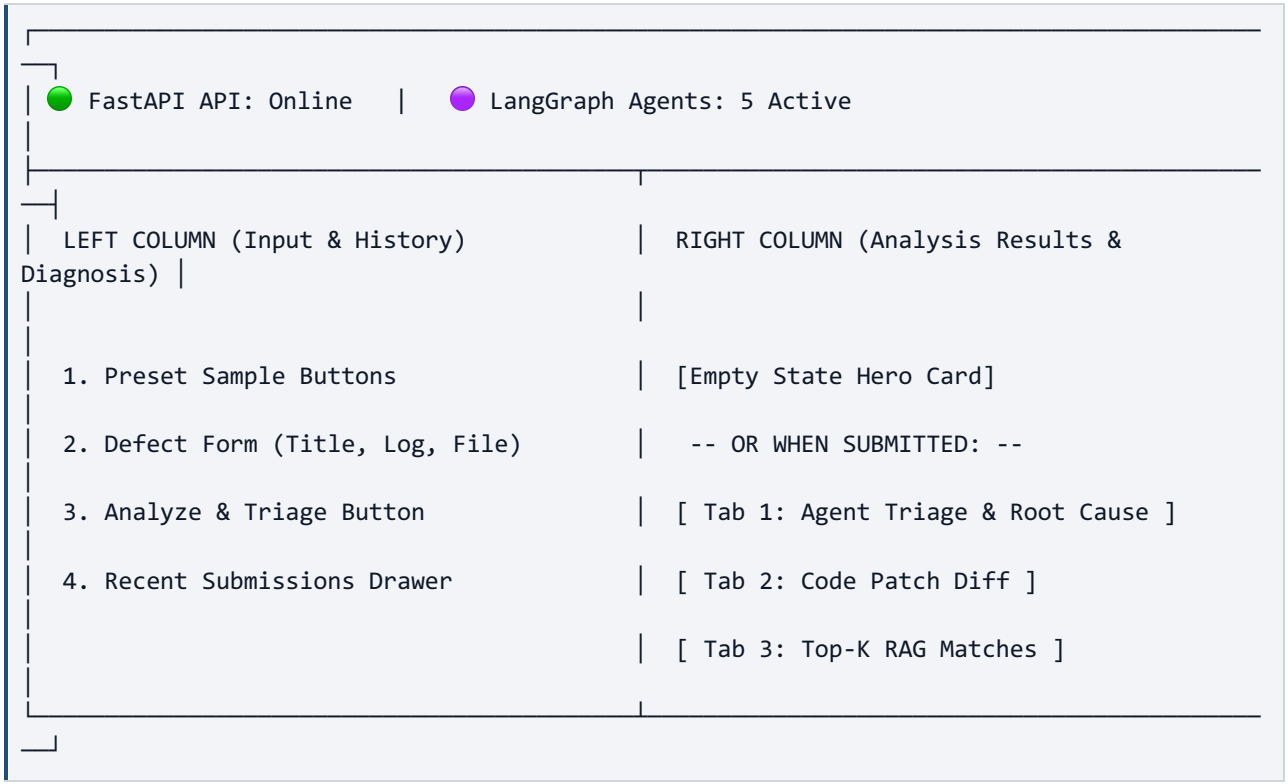
DefectMind UI Architecture & Workflow Guide

Comprehensive Technical & Visual Walkthrough of the Next.js 14 Dashboard

1. Dashboard Layout Overview

The DefectMind User Interface is built with Next.js 14 (App Router), TypeScript, and Tailwind CSS. It features a responsive, dark-mode glassmorphic interface designed around a dual-column layout:

- Header Status Bar: Displays live backend health status (FastAPI endpoint status) and active LangGraph agent counters.
- Left Column (5 Cols): Contains the defect submission form, preset crash log loaders, drag-and-drop file uploader, and submission history drawer.
- Right Column (7 Cols): Displays either the empty state hero card or the interactive 3-tab analysis view (Triage Diagnosis, Code Remediation Diff, and Historical RAG Matches).



2. Step-by-Step UI Workflow

Step 1: Inputting a Defect

- Click a Sample Preset: Top preset buttons (Apache Kafka OOM, Mozilla Memory Leak, Eclipse Compiler Lock) instantly populate sample title, trace, environment, repository, and component fields.

- Type or Paste Details: Enter a custom defect title, select target repository and component, paste raw stack traces or log dumps.
- Drag & Drop Log Files: Drag and drop .log, .txt, or .json log files directly into the file attachment area.

Step 2: Submitting & Triggering the Multi-Agent Engine

- Click 'Analyze & Triage Defect' to dispatch the multipart form data to FastAPI (/api/v1/bugs/submit).
- The button displays an active loading spinner while the 5-agent LangGraph workflow executes (Log Analysis -> Duplicate Check -> RAG Vector Retrieval -> Triage -> Remediation).
- Upon response, the main page state updates to display the analysis report and appends the item to the history drawer.

Step 3: Tab 1 - Agent Triage & Root Cause Diagnosis

- Triage Badges: Visual Severity (Blocker, Major, Critical) and Priority (P1-P5) indicator pills.
- Confidence Gauge: An animated percentage meter reflecting root-cause diagnosis confidence.
- De-noised Stack Frame Isolation: Highlights the isolated exception type, message, and top offending file & line number.
- LangGraph Orchestration Log: Timeline audit trail detailing execution steps of each AI agent.

Step 4: Tab 2 - Code Remediation & Patch Diff Viewer

- Unified Diff Patch: Syntax-highlighted git diff display with additions in green, deletions in red, and hunk headers in indigo.
- Copy Patch Button: One-click clipboard copy functionality.
- Mitigation Checklist: Step-by-step resolution advice for engineers.

Step 5: Tab 3 - Top-K RAG Vector Matches

- ChromaDB Matches: Displays historical defect matches from Mozilla, Apache, and Eclipse datasets.
- Similarity Badges: Percentage match pills (e.g. 85% Match).
- Expandable Accordion Cards: Click to view historical descriptions and verified fix patches.

Step 6: Submission History Drawer

- Displays recent bug submissions. Clicking any historical submission instantly re-populates the right panel with its analysis report.

3. Code Location Reference Table

Component File	Role & Description	Key Code References
`src/app/page.tsx`	Main Dashboard Entry Point	`Header Bar (L47-77), Global State (L20-33), Dual Panel Layout (L80-220)`

src/components/BugSubmissionForm.tsx	Defect Submission Form	Presets (L13-41), Drag-Drop Upload (L69-76), Submit Handler (L93-120)
src/components/DiagnosisView.tsx	Triage & Root Cause Panel	Triage Badges (L71-80), Confidence Gauge (L96-110), Agent Timeline (L152-176)
src/components/CodeRemediationDiff.tsx	Unified Code Patch Viewer	Diff Highlighter (L22-38), Copy Button (L56-71), Mitigation Steps (L80-91)
src/components/SimilarBugsList.tsx	Historical RAG Matches	Match Badges (L77-87), Expandable Accordions (L98-119)
src/lib/api.ts	Axios API Client & Interfaces	submitBugReport (L78-85), getSubmissionsList (L92-95), Type Definitions (L5-63)